

Anatomie d'un dossier Run/

que contient chaque sous-dossier, fichier par fichier

DeepEdge Benchmark · état au 8 juillet 2026

1. À quoi sert le dossier Run/

Run/ est l'entrepôt des résultats du benchmark. À chaque fois qu'un modèle est entraîné et testé sur un actif et un horizon, le pipeline crée **un sous-dossier dédié** qui contient tout ce qui décrit ce run : les données utilisées, le modèle entraîné, ses performances, sa prévision finale et ses tests. Rien n'est écrasé : chaque combinaison a son dossier.

Il y a **50 sous-dossiers** = 5 modèles × 5 actifs × 2 horizons. Le nom d'un dossier encode ces quatre informations.

Comment lire le nom d'un dossier

Run/20260707-ARIMA-BTC-USD-D1/

Morceau	Signification
20260707	date du run (7 juillet 2026)
ARIMA	le modèle (ARIMA, SARIMA, Prophet, LSTM, Naive)
BTC-USD	l'actif prédit
D1	l'horizon : D1 = 1 jour, D7 = 7 jours

2. Vue d'ensemble des fichiers

Les fichiers d'un dossier se rangent en quatre familles. Le schéma ci-dessous les regroupe par rôle.

Le contexte du run

metadata.json
hyperparams.json

actif, fenêtre de données, versions des libs, commit git, seed
hyperparamètres du modèle (ordre ARIMA, epochs LSTM...)

Les données

prices.parquet
predictions.parquet
residuals.parquet

série de prix d'entrée : date + close
backtest : date · actual · predicted · pi_lower · pi_upper
résidus du modèle (ARIMA / SARIMA uniquement)

Les résultats

metrics.json
forecast.json
business_validation.json

performance backtest : RMSE, MAE, MAPE, dir. acc, couv. IC 95 %
LA prévision finale : point + intervalle pour l'horizon
la ligne envoyée à tracking.db (record de validation métier)

Le modèle & les tests

model.pkl / .json / .h5
scaler.pkl
unit_tests.json

le modèle entraîné, sérialisé (format selon le modèle)
normaliseur des données (LSTM uniquement)
résultat des tests unitaires du modèle pour ce run

50 dossiers au total = 5 modèles × 5 actifs × 2 horizons

3. Chaque fichier en détail

Le contexte du run

Fichier	Contenu
metadata.json	La carte d'identité du run. Actif et classe d'actif, fréquence, fenêtre de données (window_start → window_end), date de fin d'entraînement, date du run, commit git, seed aléatoire, et les versions exactes de toutes les librairies (numpy, pandas, statsmodels, tensorflow...). C'est ce qui rend le run reproductible.
hyperparams.json	Les réglages du modèle. Varie selon le modèle : ordre (p,d,q) et paramètres GARCH pour ARIMA ; longueur de séquence, units, epochs pour LSTM ; échelles de saisonnalité pour Prophet. Le modèle Naive n'en a pas.

Les données

Fichier	Contenu
prices.parquet	La série de prix d'entrée. Deux colonnes : date et close. C'est l'historique brut (~750 points, soit 3 ans) qui alimente le modèle.
predictions.parquet	Le backtest. Cinq colonnes : date, actual (vrai prix), predicted

Fichier	Contenu
	(prévision), pi_lower et pi_upper (intervalle). Une ligne par jour de la période de validation — c'est la matière des métriques.
residuals.parquet	Les résidus. L'écart entre prévision et réalité sur l'entraînement, utile pour diagnostiquer le modèle. Présent uniquement pour ARIMA et SARIMA.

Les résultats

Fichier	Contenu
metrics.json	Les performances chiffrées du backtest. RMSE, MAE, MAPE (erreurs), directional_accuracy (% de bons sens de variation), pi_coverage_95 (% de fois où le vrai prix tombe dans l'intervalle), largeur de l'intervalle (min/moyenne/max), et n_val (nombre de points testés).
forecast.json	LA prévision finale. Le résultat qui compte : dernière date connue, dernier prix, valeur prédite pour l'horizon, et bornes de l'intervalle (pi_lower / pi_upper). Court et précis.
business_validation.json	La ligne de validation métier. Exactement le record envoyé à la base tracking.db (mêmes champs : y_pred, intervalle, verdicts, et après coup y_true et les métriques d'évaluation). C'est le pont entre Run/ et la base de suivi.

Le modèle entraîné et ses tests

Fichier	Contenu
model.pkl / .json / .h5	Le modèle sérialisé, prêt à recharger. Le format dépend du modèle : .pkl pour ARIMA et SARIMA, .json pour Prophet, .h5 pour le réseau LSTM. Permet de rejouer une prévision sans ré-entraîner.
scaler.pkl	Le normaliseur. Sauvegarde l'échelle utilisée pour ramener les prix entre 0 et 1 avant de nourrir le réseau. Présent uniquement pour LSTM.
unit_tests.json	Le rapport de tests du modèle. Statut global (pass/fail), résumé pytest, et la liste des tests exécutés avec leur résultat. Sert de preuve que le modèle a passé ses tests au moment du run. (Détailé dans le document sur les tests unitaires.)

4. Quels fichiers selon le modèle

Tous les dossiers partagent le même socle (contexte, données de prix, résultats, tests). Ce qui change, c'est le format du modèle et quelques fichiers spécifiques.

Fichier	ARIMA	SARIMA	Prophet	LSTM	Naive
metadata.json	✓	✓	✓	✓	✓
hyperparams.json	✓	✓	✓	✓	—
metrics.json	✓	✓	✓	✓	✓

Fichier	ARIMA	SARIMA	Prophet	LSTM	Naive
forecast.json	✓	✓	✓	✓	✓
business_validation.json	✓	✓	✓	✓	✓
unit_tests.json	✓	✓	✓	✓	✓
prices.parquet	✓	✓	✓	✓	—
predictions.parquet	✓	✓	✓	✓	—
residuals.parquet	✓	✓	—	—	—
model (pkl/json/h5)	✓	✓	✓	✓	—
scaler.pkl	—	—	—	✓	—

Le modèle Naive (« demain = aujourd'hui ») n'a ni hyperparamètres, ni données/backtest, ni modèle à sérialiser : il sert uniquement de référence de comparaison.

En résumé

- Un dossier Run/ par combinaison modèle × actif × horizon (50 au total).
- Le nom encode date, modèle, actif, horizon.
- Quatre familles de fichiers : contexte, données, résultats, modèle & tests.
- Le fichier forecast.json porte la prévision finale ; business_validation.json est le lien vers tracking.db.
- Format du modèle selon le type ; Naive n'a qu'un socle minimal.